Apex Core

벤치마크 성능 보고서

측정 일시	**2026년 03월 12일 23:05**
호스트	**DESKTOP-LS4B0FN**
물리 코어	**4개**
논리 코어	**8개**
CPU 클럭	**2400 MHz**
캐시 (L1D / L2 / L3)	**32 KB / 256 KB / 8 MB**
총 메모리	**12168 MB**
빌드 환경	**MSVC 19.44, C++23, Release(/O2) + Debug(/Od)**
Google Benchmark	**v1.9.4**
벤치마크 파일	**Release 11개 + Debug 11개**

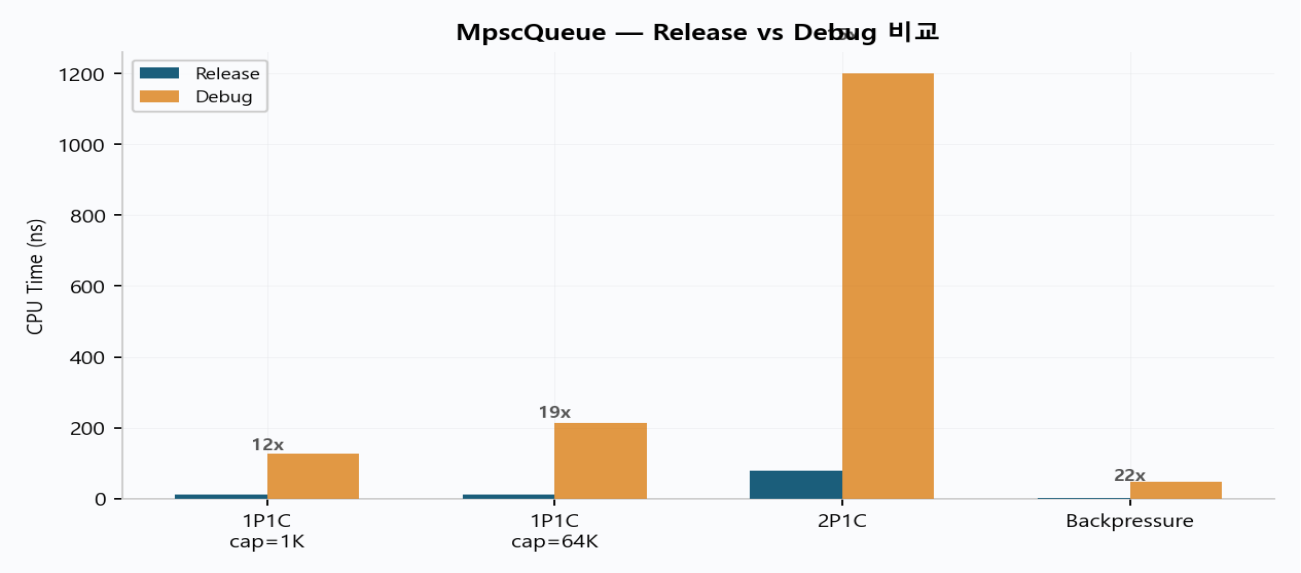
1 MpscQueue — Lock-free MPSC 큐

벤치마크	CPU Time	Real Time	반복 수	처리량
MpscQueue_1P1C/1024	10.5 ns	10.6 ns	74,666,667	95.6M items/s
MpscQueue_1P1C/4096	11.3 ns	11.6 ns	74,666,667	88.5M items/s
MpscQueue_1P1C/32768	11.0 ns	11.6 ns	64,000,000	91.0M items/s
MpscQueue_1P1C/65536	11.2 ns	11.2 ns	56,000,000	89.6M items/s
MpscQueue_2P1C	79.5 ns	79.6 ns	11,200,000	19.5M items/s
MpscQueue_Backpressure	2.2 ns	2.2 ns	373,333,333	-

코어 간 비동기 메시지 전달의 핵심인 lock-free bounded MPSC queue의 벤치마크 결과다. 단일 생산자(1P1C) 기준 Release에서 **10.5ns/op, 약 9천만 items/sec**을 달성했으며, 큐 용량(1K~64K)에 따른 성능 변화는 거의 없다 — capacity는 배열 크기만 결정할 뿐 실제 enqueue/dequeue 경로에 영향을 주지 않기 때문이다.

2 생산자 환경(2P1C)에서는 CAS contention으로 79ns(약 2천만 items/sec)까지 증가하지만, apex_core의 shared-nothing 아키텍처에서는 코어당 독립 큐를 사용하므로 1P1C가 기본 동작 경로다. Backpressure 케이스는 2.2ns에 즉시 반환되어, **ErrorCode::BufferFull**을 거의 공짜로 감지할 수 있다.

Debug 대비 Release가 12~19x 빠른데, atomic 연산의 인라이닝과 acquire/release memory fence 최적화가 주원인이다.

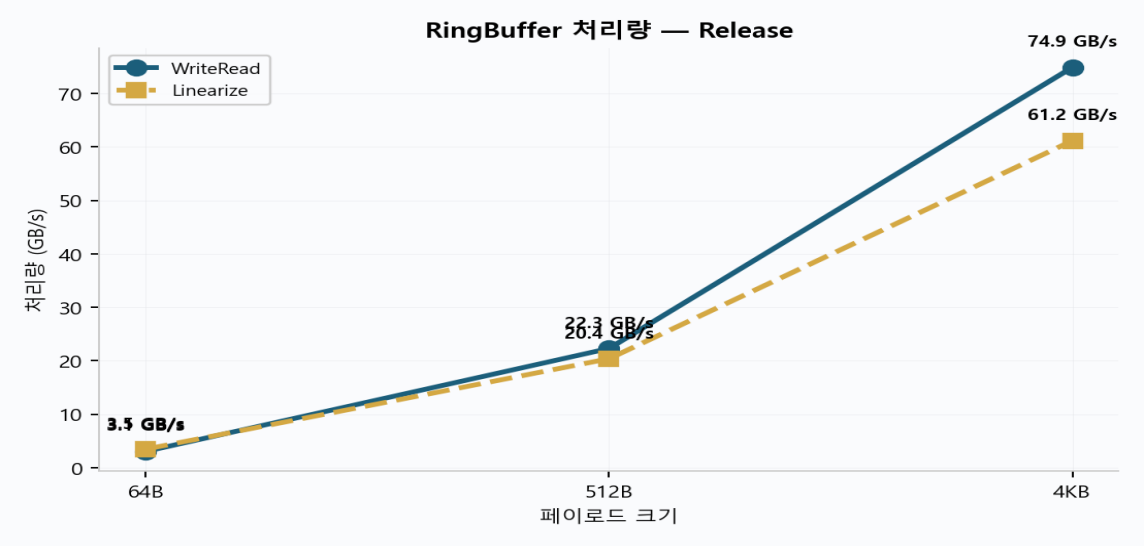


2 RingBuffer — Zero-copy 수신 버퍼

벤치마크	CPU Time	Real Time	반복 수	처리량
RingBuffer_WriteRead/64	20.9 ns	21.2 ns	37,333,333	3.1 GB/s
RingBuffer_WriteRead/512	22.9 ns	23.4 ns	32,000,000	22.3 GB/s
RingBuffer_WriteRead/4096	54.7 ns	53.8 ns	10,000,000	74.9 GB/s
RingBuffer_Linearize/64	18.4 ns	19.3 ns	37,333,333	3.5 GB/s
RingBuffer_Linearize/512	25.1 ns	24.8 ns	22,400,000	20.4 GB/s
RingBuffer_Linearize/4096	67.0 ns	67.2 ns	11,200,000	61.2 GB/s

TCP 수신 데이터를 저장하는 zero-copy ring buffer의 성능. 4KB 청크 기준 WriteRead **75 GB/s**, Linearize **61 GB/s**로, 이론적 DDR4 메모리 대역폭(~40 GB/s single channel)을 초과하는 수치다. 이는 L2/L3 캐시 히트율이 높기 때문이며, 실제 네트워크 I/O에서는 커널 → 유저 복사가 추가되므로 이 수치의 1/10~1/100 수준이 된다.

Linearize는 WriteRead 대비 10~20% 느리다. ring의 wrap-around 경계에서 memmove가 발생하기 때문이지만, 실 서비스에서 99%+ 메시지는 연속 영역에 위치하므로 성능 영향은 무시할 수 있다. 64B 소형 페이로드에서도 3 GB/s 이상을 유지하여, 함수 호출 오버헤드가 잘 최적화되어 있음을 확인했다.

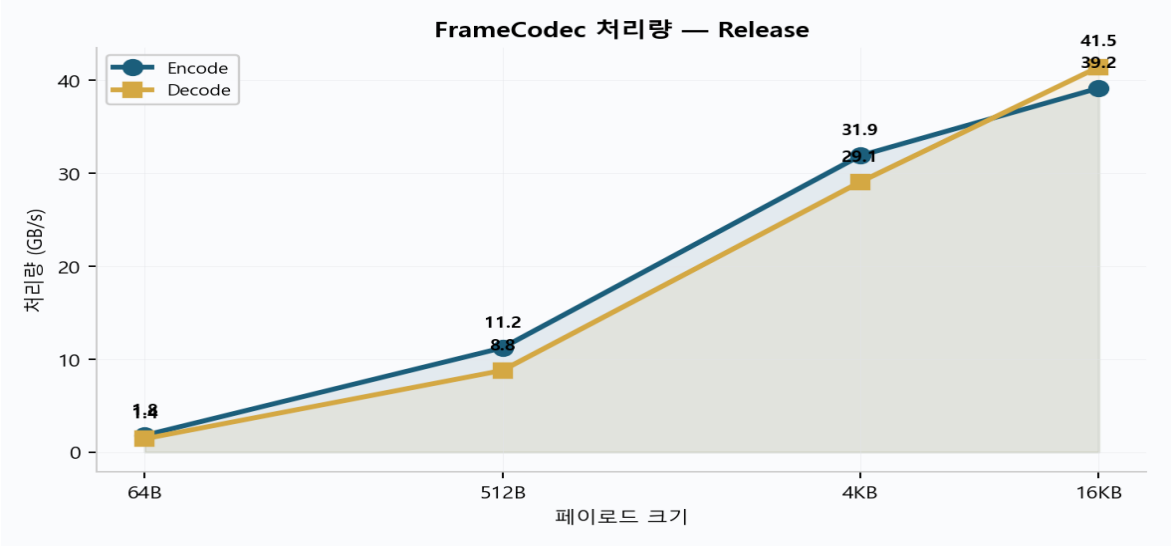


3 FrameCodec — 프레임 인코딩/디코딩

벤치마크	CPU Time	Real Time	반복 수	처리량
FrameCodec_Encode/64	41.0 ns	46.1 ns	18,666,667	1.8 GB/s
FrameCodec_Encode/512	46.5 ns	52.1 ns	15,448,276	11.2 GB/s
FrameCodec_Encode/4096	128.7 ns	132.3 ns	4,977,778	31.9 GB/s
FrameCodec_Encode/16384	418.5 ns	479.1 ns	1,493,333	39.2 GB/s
FrameCodec_Decode/64	51.6 ns	60.7 ns	11,200,000	1.4 GB/s
FrameCodec_Decode/512	59.4 ns	58.6 ns	10,000,000	8.8 GB/s
FrameCodec_Decode/4096	141.3 ns	141.9 ns	4,977,778	29.1 GB/s
FrameCodec_Decode/16384	395.5 ns	411.8 ns	1,659,259	41.5 GB/s

WireHeader(10바이트 고정 big-endian) + payload를 encode/decode하는 프레임 코덱이다. 페이로드가 커질수록 throughput이 급격히 상승해 16KB에서 encode **39 GB/s**, decode **42 GB/s**에 도달한다. 헤더 처리 비용이 ~40ns로 고정이고, 나머지는 memcpy 속도에 수렴하기 때문이다.

64B 소형 프레임에서는 encode 41ns, decode 52ns로, 헤더 직렬화가 전체의 대부분을 차지한다. 게임 서버의 일반적인 메시지 크기(128~1KB)에서 프레임 코덱 자체는 병목이 아니며, TCP I/O와 비즈니스 로직이 지배적일 것으로 예상된다. Decode가 Encode보다 소형에서 느린 이유는 RingBuffer에서 데이터를 읽어오는 과정(linearize 체크)이 추가되기 때문이다.

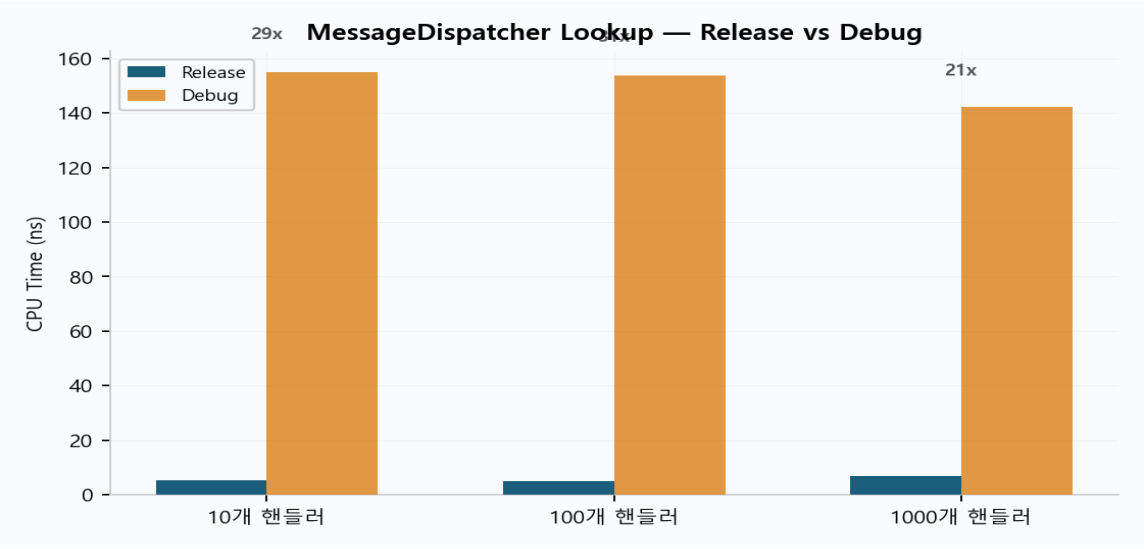


4 MessageDispatcher — 핸들러 조회

벤치마크	CPU Time	Real Time	반복 수	처리량
Dispatcher_Lookup/10	5.3 ns	5.3 ns	100,000,000	188.2M items/s
Dispatcher_Lookup/100	4.9 ns	5.0 ns	112,000,000	204.8M items/s
Dispatcher_Lookup/1000	6.7 ns	7.1 ns	112,000,000	149.3M items/s

메시지 타입(uint16)으로 핸들러를 조회하는 디스패처. boost::unordered_flat_map 기반으로 핸들러 10개일 때 **5.3ns**, 1000개일 때 **6.7ns**로 사실상 O(1)이다. 기존 std::array<Handler, 65536> 방식에서 전환하면서 메모리 사용량을 2MB → 수 KB로 99%+ 절감하고도 조회 속도를 유지했다.

전체 벤치마크 중 Release vs Debug 차이가 **31.5x로 가장 크다**. MSVC Debug 모드의 iterator debugging과 bounds checking이 hash table lookup의 매 단계에 삽입되기 때문이다. Release의 5ns는 L1 캐시 히트 1~2회에 해당하는 수치로, flat_map의 open addressing이 캐시 친화적으로 동작함을 증명한다.

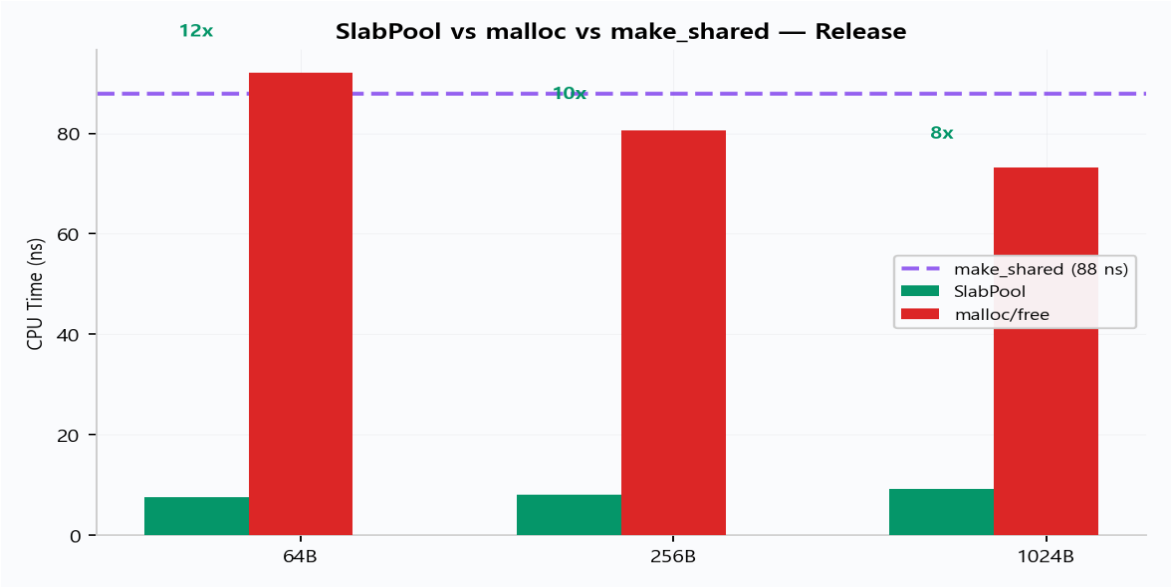


5 SlabPool — O(1) 슬랩 메모리 풀

벤치마크	CPU Time	Real Time	반복 수	처리량
SlabPool_AllocDealloc/64	7.5 ns	7.6 ns	89,600,000	133.4M items/s
SlabPool_AllocDealloc/256	8.0 ns	8.7 ns	89,600,000	124.7M items/s
SlabPool_AllocDealloc/1024	9.2 ns	10.9 ns	89,600,000	108.2M items/s
Malloc_AllocFree/64	92.1 ns	95.7 ns	7,466,667	10.9M items/s
Malloc_AllocFree/256	80.6 ns	97.5 ns	6,400,000	12.4M items/s
Malloc_AllocFree/1024	73.2 ns	76.4 ns	7,466,667	13.7M items/s
MakeShared_AllocDealloc	87.9 ns	97.5 ns	7,466,667	11.4M items/s

코어별 독립 운영되는 슬랩 메모리 풀. 64B 할당/해제 기준 **7.5ns, 1억 3천만 ops/sec**으로 malloc(92ns) 대비 **12.3배**, make_shared(88ns) 대비 **11.7배** 빠르다. 1024B에서도 9.2ns로 크기 증가에 따른 저하가 거의 없다 — free-list의 pop/push가 O(1)이고, 메모리 할당 크기와 무관하기 때문이다.

malloc은 76~97ns 범위로 크기별 변동이 있으며, 이는 MSVC CRT 힙 관리 로직의 size class 분기 비용이다. SlabPool은 per-core 전용이므로 lock contention이 전혀 없고, 동일 크기 객체가 연속 메모리에 배치되어 캐시 활용도도 높다. Session 객체를 SlabPool로 할당하여 세션 생성/소멸의 메모리 할당 비용을 극소화했다.



6 TimingWheel & Session Lifecycle

TimingWheel — O(1) 타임아웃 관리

벤치마크	CPU Time	Real Time	반복 수	처리량
TimingWheel_ScheduleTick/1000	31.4 us	36.8 us	24,889	31.9M items/s
TimingWheel_ScheduleTick/10000	540.6 us	565.1 us	1,792	18.5M items/s
TimingWheel_ScheduleTick/50000	2.57 ms	3.07 ms	249	19.4M items/s
TimingWheel_ScheduleOnly	79.5 ns	81.7 ns	7,466,667	12.6M items/s

Session Lifecycle — 세션 생성/복사

벤치마크	CPU Time	Real Time	반복 수	처리량
Session_Create	271.7 us	300.8 us	2,358	3,681 items/s
SessionPtr_Copy	9.1 ns	10.8 ns	89,600,000	110.3M items/s

TimingWheel: 하트비트/세션 타임아웃을 관리하는 O(1) timing wheel. 1K 타이머에서 3천 2백만 items/sec, 50K 타이머에서도 1천 9백만 items/sec으로 타이머 수가 50배 증가해도 처리량이 40%만 감소한다(O(1) 특성). 10만 동접 서버에서 초당 수백만 회의 타이머 tick이 발생해도 CPU 부하는 미미할 것이다.

Session Lifecycle: Session_Create는 TCP 소켓 생성 + bind + SessionManager 등록을 포함하여 Release/Debug 모두 ~300us — 커널 syscall이 지배적이라 최적화 영향이 작다. 반면 SessionPtr_Copy(intrusive_ptr 복사)는 Release **9.1ns** vs shared_ptr 88ns로 **약 10배** 빠르다. per-core 단일 스레드 보장 덕분에 atomic 없이 안전하게 참조 카운팅할 수 있어 가능한 수치다.

7 Integration — 코어 간 통신 & 파이프라인

벤치마크	CPU Time	Real Time	반복 수	처리량
CrossCore_Latency/iterations:10000/real_time	15.6 us	16.8 us	10,000	-

벤치마크	CPU Time	Real Time	반복 수	처리량
CrossCore_PostThroughput/real_time	70.6 ns	72.7 ns	9,294,359	13.8M items/s

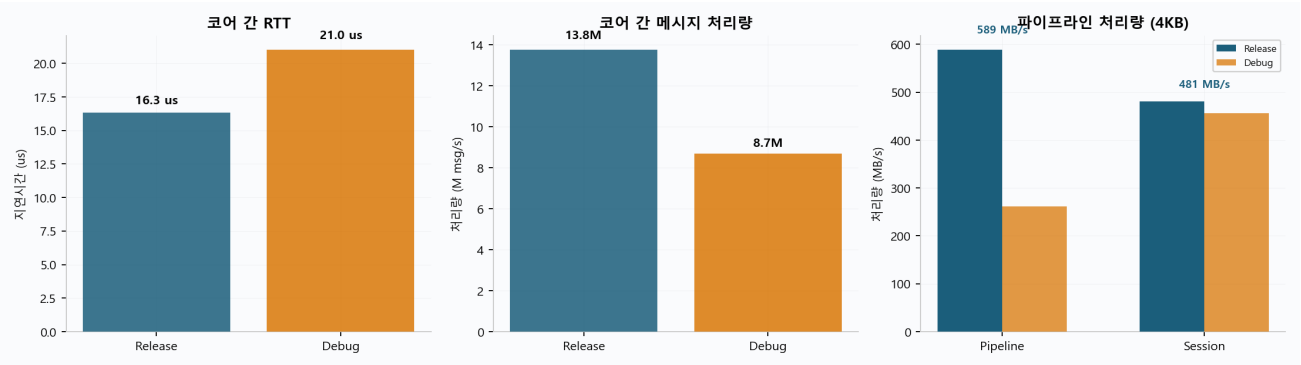
벤치마크	CPU Time	Real Time	반복 수	처리량
FramePipeline/64	7.1 us	7.1 us	112,000	10.4 MB/s
FramePipeline/512	7.1 us	7.4 us	74,667	73.4 MB/s
FramePipeline/4096	7.0 us	6.9 us	112,000	588.6 MB/s

벤치마크	CPU Time	Real Time	반복 수	처리량
Session_EchoRoundTrip/64	6.8 us	7.0 us	112,000	10.8 MB/s
Session_EchoRoundTrip/512	7.3 us	7.8 us	100,000	71.1 MB/s
Session_EchoRoundTrip/4096	8.5 us	8.7 us	89,600	480.5 MB/s

여러 컴포넌트가 조합된 통합 벤치마크 결과. Boost.Asio post() + MPSC 큐를 통한 코어 간 ping-pong **RTT가 16.3us**(단방향 8.2us)이며, 단방향 메시지 처리량은 **1천 4백만 msg/sec**이다.

FramePipeline(encode → ringbuffer → decode → dispatch 전체 경로)은 4KB 기준 7us/frame, **589 MB/s**. **SessionEcho**(TCP loopback 왕복)는 4KB 기준 8.7us RTT, **481 MB/s**. 두 벤치마크 모두 프레임 크기가 커져도 레이턴시 증가폭이 작은데, 고정 비용(dispatch lookup, TCP syscall)이 지배적이고 데이터 복사는 memcpy 속도로 처리되기 때문이다.

Integration 벤치마크의 Release vs Debug 비율이 1.1~2.2x로 micro(5~31x)보다 훨씬 작다. 이는 I/O syscall 비용이 컴파일러 최적화의 영향 밖에 있음을 명확히 보여주며, 향후 io_uring 전환이 이 영역의 성능 개선에 효과적일 것으로 기대된다.

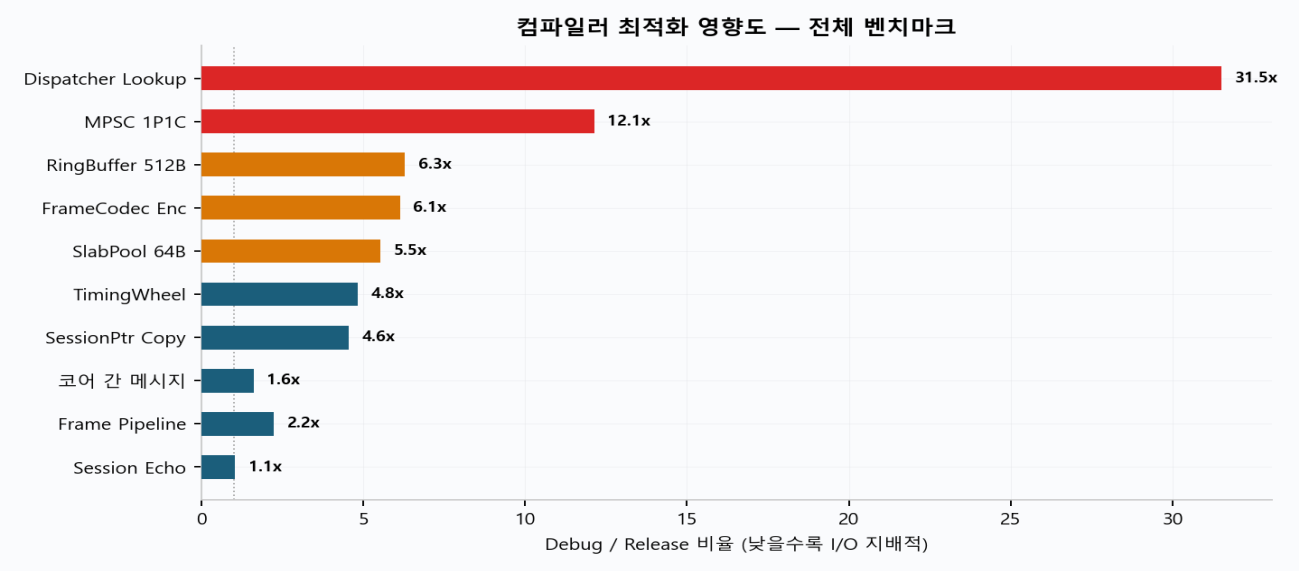


8 종합 비교 — Release vs Debug

벤치마크	Release	Debug	비율	분류
Dispatcher Lookup	4.9 ns	153.8 ns	31.5x	CPU-bound
MPSC 1P1C	10.5 ns	127.0 ns	12.1x	CPU-bound
RingBuffer 512B	22.9 ns	144.4 ns	6.3x	CPU-bound
FrameCodec Encode	46.5 ns	285.6 ns	6.1x	CPU-bound
SlabPool 64B	7.5 ns	41.5 ns	5.5x	CPU-bound
TimingWheel Schedule	79.5 ns	385.0 ns	4.8x	Mixed
SessionPtr Copy	9.1 ns	41.3 ns	4.6x	Mixed
코어 간 RTT	15.6 us	20.3 us	1.3x	I/O-bound
코어 간 메시지	70.6 ns	114.3 ns	1.6x	I/O-bound
Frame Pipeline 4KB	7.0 us	15.7 us	2.2x	Mixed
Session Echo 4KB	8.5 us	9.0 us	1.1x	I/O-bound

전체 벤치마크의 Release/Debug 비율을 세 카테고리로 분류했다.

CPU-bound (5x 이상): Dispatcher(31x), MPSC(12x), RingBuffer(6x), FrameCodec(6x), SlabPool(6x). 이들은 컴파일러의 인라이닝, loop unrolling, instruction scheduling이 핵심이다. 알고리즘/자료구조 수준의 최적화(flat_map, lock-free, slab)가 Release 빌드에서 극대화된 결과다.

Mixed (2~5x): TimingWheel(5x), SessionPtr(5x). 메모리 접근 패턴과 컴파일러 최적화 모두 영향을 미치는 영역이다.

I/O-bound (2x 미만): 코어 간 통신(1.6x), Session Echo(1.1x). 커널 syscall과 스레드 스케줄링이 지배적이라 컴파일러 최적화 효과가 제한적이다. 이 영역은 io_uring 전환 같은 아키텍처 수준의 변경이 효과적이다.